

Genomic Sequence Matching Engine

Under The Supervision of
Eng. Ahmed Hussein
Eng. Mohammed Salah

Group: S26-B7-FPGA-G7

Team Members

Alaa Maher Eldeeb - 213584
Nada Haron - 219628
Sarah Samir Ali - 232384
Rofida Mahmoud - 301873
Bassma Mohamed - 345332

Project Overview

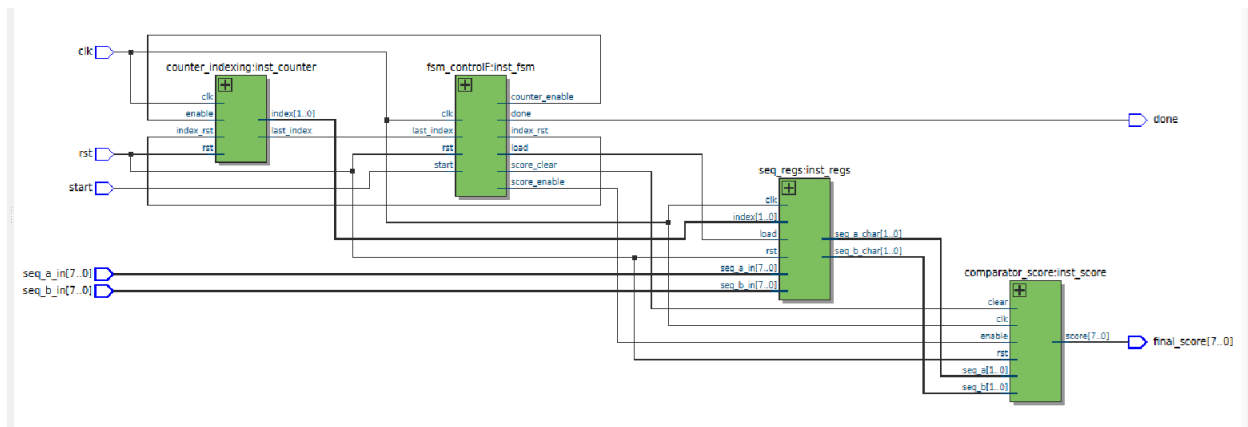
Sequence comparison is an important task in genomic analysis, but it can require significant computational time. This project develops a hardware-based DNA sequence matching engine using Verilog to speed up the comparison process. The design uses a 2-bit encoding scheme, where A=00, C=01, G=10, and T=11. The engine processes synthetic DNA sequences character by character and calculates the alignment score using a custom hardware datapath. This provides a fast and deterministic way to compare DNA sequences.

Motivation

The reason we have selected this project is because sequence comparison is an essential activity in the field of bioinformatics, but the process is slow if you do it entirely through software, especially when multiple sequences have to be compared. Therefore, we thought of designing a hardware-based sequence comparator that could compare two DNA sequences and calculate scores faster. This project will provide us with an opportunity to put knowledge of various digital design principles such as FSMs, counters, registers, and combinational and sequential logics into practice.

System Architecture

The architecture consists of four RTL functional blocks instantiated in the top level wrapper (genomic_matcher_top). The design is a synchronous iterative architecture controlled by an FSM. Each clock cycle, the current sequence is selected, updated and the score is updated. The operation resumes until the final position is compared.



Block	RTL module	Primary role
Control	fsm_control	- Control the operation using IDLE, COMPARE, DONE. - Generate the control pulses for loading the sequence, enable & reset the counter, clear & enable the score and asserting done
Sequence storage	seq_regs	Store the two input sequences and show the nucleotide at the current_index.
Index counter	counter_indexing	Tracks the current nucleotide position and generates last_index when the final position is reached.
Comparison & Scoring	comparator_score	Compares the two selected nucleotides and updates the accumulated score: +1 or -1

Top-Level Integration:

Serves as the main structural wrapper that connects the sequence registers, counter, comparator ALU, and FSM control unit. It connects the internal control signals, such as load, counter_enable, and score_clear, between the modules. It provides the testbench with only the required inputs and outputs: the clock, reset, start signal, packed sequences (seq_a_in and seq_b_in), the final score, and the done signal.

Datapath architecture

The datapath does the main computation while FSM controls the system. The operation to select one pair compares them, updates the score then move to the next index.

1. Sequence storage:

seq_regs is the storage element for the two inputs. Each input sequence is represented as array of CHAR_WIDTH. When loading, for loop store the inputs into the corresponding array element. The outputs are seq_a char = seq_a [index] and same for B. so the index here which nucleotide pair gonna be compared.

2. index counter

The counter uses the sequence length to get parameterized and the log of the length bits for the index. The index reset to 0 when using rst or index_rst. During COMPARE control the counter_enable allows the index to be increased until we reach the final index. The last index closes the loop, telling the FSM when the comparing cycle reach the final position.

3. Comparator and Accumulator

On each comparison cycle, the score block do a direct equality test. The score increased by one when find a match otherwise the score gets decreased. The score is signed, so we can detect a negative mismatch. The range of the score for N sequence length goes from -N to +N. Because our

Control Architecture : FSM

Fsm_control is a three-state Moore controller. The states are IDLE,COMPARE and DONE. The “start” forces the controller to leave IDLE, “last_index” forces it to leave COMPARE and we return to IDLE after DONE on the next clock edge.

State	usage	controls	Exit condition
IDLE	Waiting for a new transaction.	If start=1: load=1, index_rst=1, score_clear=1	start sampled high -> COMPARE
COMPARE	Iteratively score the position selected by current_index.	score_enable=1, counter_enable=1	last_index=1 -> DONE
DONE	Transaction complete; result is available.	done=1	Unconditional -> IDLE

IDLE is the waiting state, COMPARE performs the comparison operation, and DONE indicates that the final result is ready.

The FSM doesn't perform the comparison itself; it controls the Counter, Registers, and Score unit through control signals.

The Counter generates last_index when the current index reaches the last position. The FSM then moves from COMPARE to DONE.

A useful architectural detail is that the control outputs are combinationaly decoded from the current state, while the state itself is clocked. Therefore, when start is high while the controller is still in IDLE, load, index_rst, and score_clear are asserted during that cycle. At the active edge, the input sequences, index, and score are initialized and the FSM advances into COMPARE. This establishes a clean transaction boundary.

1. State transition logic
2. Control signal meaning

Signal	Source	Destination	Function
load	FSM	seq_regs	Capture both packed input sequences into the internal arrays.
index_rst	FSM	counter_indexing	Set the current comparison position back to zero.
counter_enable	FSM	counter_indexing	Advance the sequence position during COMPARE.
score_clear	FSM	comparator_score	Clear the accumulated score at transaction start.
score_enable	FSM	comparator_score	Perform one match/mismatch score update per compare cycle.
last_index	counter_indexing	FSM	Indicates that current_index is the terminal sequence position.
done	FSM	top-level	Indicates that final_score is the completed result.

Cycle operation

For SEQ_LEN=4, a complete operation cycle consists of one initialization phase followed by four comparison cycles and then a DONE state. The exact simulation edge alignment depends on how start is driven by the testbench, but the logical sequence is deterministic.

transaction

Phase	Typical current state	Key events	Index/score behavior
1. Request	IDLE	start high; load, index_rst, score_clear asserted	Sequences captured; index=0; score=0
2. Compare pos 0	COMPARE	score_enable and counter_enable asserted	Compare A[0] vs B[0]; then index advances to 1

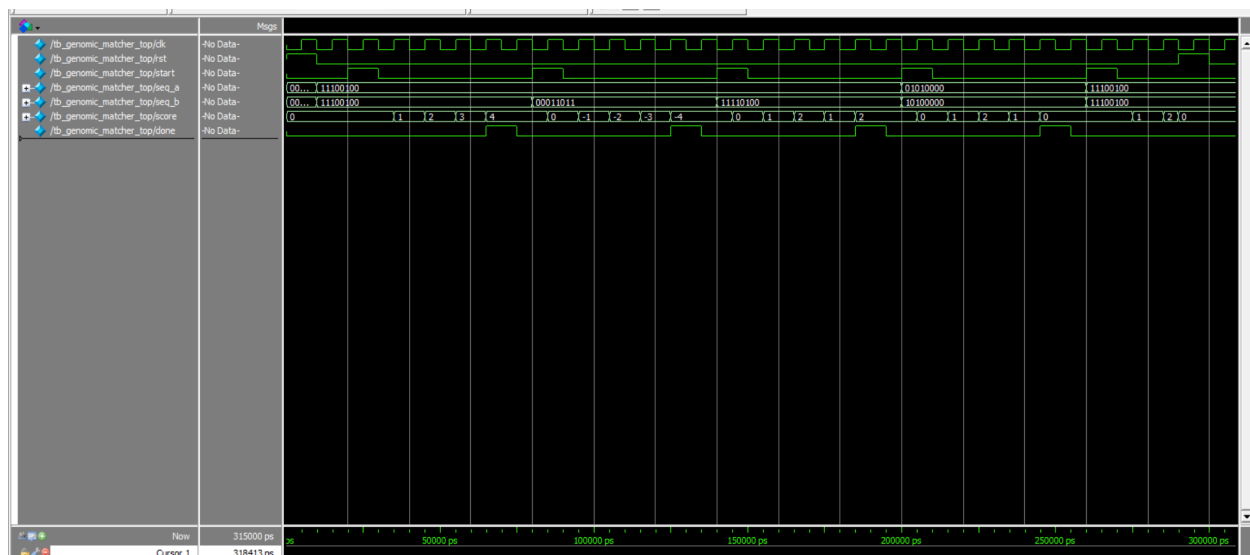
3. Compare pos 1	COMPARE	same control pattern	Compare A[1] vs B[1]; then index advances to 2
4. Compare pos 2	COMPARE	same control pattern	Compare A[2] vs B[2]; then index advances to 3
5. Compare pos 3	COMPARE	score_enable remains asserted; last_index=1	Compare A[3] vs B[3]; counter holds at 3
6. Complete	DONE	done=1	final_score is the completed accumulated value
7. Return	IDLE	done deasserts after state transition	Ready for next transaction

Verification Results

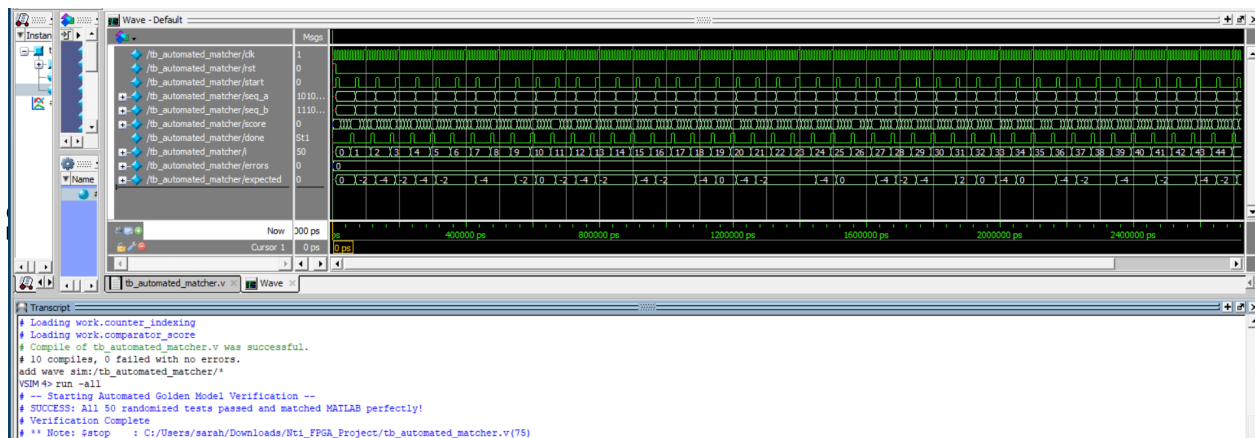
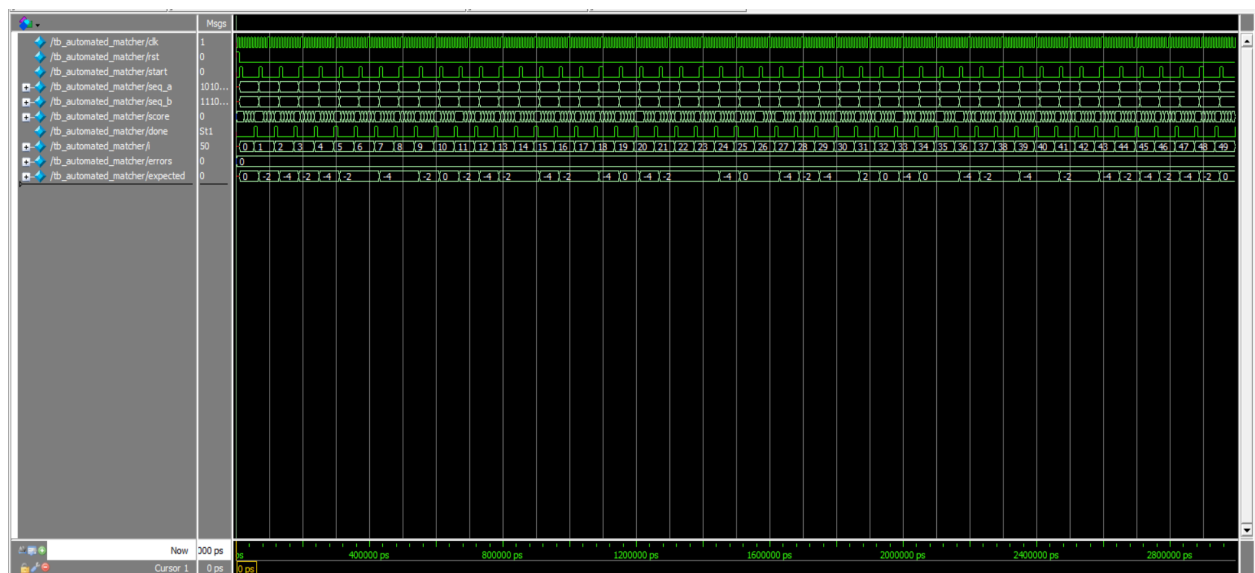
The system was tested using a self-checking Verilog testbench. To avoid timing problems between the testbench and the FSM, delays such as (#15) were used to apply inputs on the falling edge of the clock. ModelSim waveforms confirmed that the FSM and datapath worked correctly in all test cases:

- **Test 1 (All Match):** The identical sequences ACGT and ACGT produced the maximum score of +4.
- **Test 2 (All Mismatch):** The sequences ACGT and TGCA produced a score of -4, confirming correct signed 2's complement arithmetic.
- **Test 3 (Mixed):** The sequences ACGT and ACTT produced the expected score of +2.
- **Test 4 (Zero Score):** The sequences CCAA and GGAA produced a final score of 0, confirming correct handling of both matches and mismatches.
- **Test 5 (Asynchronous Reset):** Resetting the system during a comparison safely stopped the process, returned the FSM to the IDLE state, and cleared the score to 0.

WaveForm and Terminal



WaveForm and Terminal



Static Timing Analysis (STA)

Static Timing Analysis is a method for checking whether a digital circuit meets its timing requirements without running a full functional simulation. STA mathematically checks every timing path in the design over the clock period, using the delay values from the standard cell library. We used it to make sure our design operates smoothly and doesn't have timing violations.

How STA was performed

We used quartus for STA. After completing the design, the Verilog modules were synthesized and the resulting timing information was analyzed using the STA tool. The critical paths were identified, and the reported maximum delay was compared with the required clock period. Setup and hold checks were also examined to verify correct synchronous operation.

The RTL design was synthesized and analyzed using the **Intel Quartus Prime 25.1** Timing Analyze. The generated timing reports were examined for setup slack, hold slack, and critical timing paths.

<<Filter>>					
	Module Name	Elapsed Time	Average Processors Used	Peak Virtual Memory	Total CPU Time
1	Analysis & Synthesis	00:00:24	1.0	4891 MB	00:00:34
2	Fitter	00:01:02	1.0	6987 MB	00:02:07
3	Timing Analyzer	00:00:21	1.0	5266 MB	00:00:21
4	Total	00:01:47	--	--	00:03:02

Time setup:

Slow 1100mV 85C Model Setup Summary			
<<Filter>>			
	Clock	Slack	End Point TNS
1	clk	-3.420	-45.623

Time hold:

Slow 1100mV 85C Model Hold Summary			
<<Filter>>			
	Clock	Slack	End Point TNS
	clk	0.366	0.000

Worst case in setup and hold :

- ▶ **i** 332146 Worst-case setup slack is -0.654
- ▶ **i** 332146 Worst-case hold slack is 0.143

Result Analysis

The STA results show that the design has a setup timing violation, while the hold requirement is satisfied. The reported worst-case setup slack is -0.654 ns. The worst-case hold slack is $+0.143$ ns, indicating that the hold-time requirement is satisfied.

Under the Slow 1100 mV, 85°C timing model, the reported setup slack is -3.420 ns, while the hold slack is $+0.366$ ns. Therefore, the current design requires timing optimization to meet the desired clock constraint.

Despite the timing issue, the hardware resource utilization is very low: the design uses only 13 ALMs ($<1\%$) and 29 registers.

Flow Status	Successful - Thu Aug 27 01:13:14 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	genomic_matcher_top
Top-level Entity Name	genomic_matcher_top
Family	Cyclone V
Device	5CGXFC7C7F23C8
Timing Models	Final
Logic utilization (in ALMs)	13 / 56,480 (< 1 %)
Total registers	29
Total pins	28 / 268 (10 %)
Total virtual pins	0
Total block memory bits	0 / 7,024,640 (0 %)
Total DSP Blocks	0 / 156 (0 %)
Total HSSI RX PCSs	0 / 6 (0 %)
Total HSSI PMA RX Deserializers	0 / 6 (0 %)
Total HSSI TX PCSs	0 / 6 (0 %)
Total HSSI PMA TX Serializers	0 / 6 (0 %)
Total PLLs	0 / 13 (0 %)
Total DLLs	0 / 4 (0 %)

Code links :https://github.com/Sarahs611/Nti_Project

References

- Tomohiro, H. et al. (2014). *"FPGA-Accelerator for DNA Sequence Alignment Based on an Efficient Data-Dependent Memory Access Scheme."* Proceedings of the International Workshop on Highly Efficient Accelerators and Reconfigurable Technologies (HEART).
- Ali, K. et al. (2020). *"Implementation of DNA sequence alignment algorithms using FPGA."* International Journal of Advanced Computer Science and Applications.
- Needleman, S. B., & Wunsch, C. D. (1970). *"A general method applicable to the search for similarities in the amino acid sequence of a protein."* Journal of Molecular Biology, 48(3), 443–453.